primer2026

August 20, 2026

1 2026 SEMM Programming Primer Part 2

William Wang

2026-08-16

[1]:
```
# Import the packages that we will need
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

[2]:
```
# Plot styles (optional)
import seaborn as sns
sns.set(style='whitegrid', font="Times New Roman", font_scale=1.2, rc={'text.
 ↪usetex' : True}) # Use 1.2 font scale for presentations/papers, and 2.0 for␣
 ↪posters
```

2 File input/output

Read in a ground motion file.

E.x. we download a file from NGA West 2 data base. It will be an .acc file

[1]:
```
# specify file paths
path1 = "data/RSN1511_CHICHI_TCU076-N.acc"
path2 = "data/RSN1511_CHICHI_TCU076-E.acc"

# How is the file formatted?

# open file 1
with open(path1, "r") as f:
    lines = f.read().splitlines() # save each line in a list without '\n'␣
 ↪newline character
for l in lines[:10]: print(l) # show it
```

```
 PEER NGA STRONG MOTION DATABASE RECORD
 Chi-Chi Taiwan, 9/20/1999, TCU076, N
 ACCELERATION TIME SERIES IN UNITS OF G
Scale Factor:   2.80000
```

```
     18000      0.00500
  0.6300641E-03  0.6304284E-03  0.6307700E-03  0.6311186E-03  0.6314781E-03
  0.6318275E-03  0.6321285E-03  0.6324153E-03  0.6327300E-03  0.6329604E-03
  0.6331967E-03  0.6335378E-03  0.6338965E-03  0.6341767E-03  0.6343153E-03
  0.6344206E-03  0.6347191E-03  0.6350274E-03  0.6351343E-03  0.6353071E-03
  0.6357249E-03  0.6362107E-03  0.6363381E-03  0.6363000E-03  0.6366267E-03
  0.6366186E-03  0.6360947E-03  0.6362384E-03  0.6368589E-03  0.6369723E-03
```

[4]:
```
# extract dt
print(lines[3])
dt = float(lines[3].split()[-1])
print(f"dt = {dt} sec")
```

```
     18000      0.00500
dt = 0.005 sec
```

[5]:
```
# Read only the data
acc_N = np.loadtxt(path1, skiprows=4, dtype=float) # specify float type

# Could use pd.read_csv or something else to read it into a DataFrame instead␣
 ↪(useful for column headers, mixed data types in one array)


print(acc_N)
print(acc_N.shape)
# Reshape it to N x 1
acc_N = np.reshape(acc_N, (-1, 1))
print(acc_N.shape)
# remove extra dimension
acc_N = np.squeeze(acc_N)
print(acc_N.shape)
```

```
[[6.300641e-04 6.304284e-04 6.307700e-04 6.311186e-04 6.314781e-04]
 [6.318275e-04 6.321285e-04 6.324153e-04 6.327300e-04 6.329604e-04]
 [6.331967e-04 6.335378e-04 6.338965e-04 6.341767e-04 6.343153e-04]
 …
 [5.296161e-05 5.327465e-05 5.358161e-05 5.388247e-05 5.417726e-05]
 [5.446596e-05 5.474857e-05 5.502510e-05 5.529552e-05 5.555981e-05]
 [5.581800e-05 5.607011e-05 5.631612e-05 5.655605e-05 5.678991e-05]]
(3600, 5)
(18000, 1)
(18000,)
```

[6]:
```
# Put it into a method
def load_gm(path: str) -> "returns the N x 1 np array and the dt":
    with open(path, "r") as f:
        lines = f.read().splitlines() # save each line in a list without '\n'␣
 ↪newline character
```

```
    # extract dt
    dt = float(lines[3].split()[-1])
    print(f"dt = {dt} sec")

    # read file
    acc = np.loadtxt(path, skiprows=4, dtype=float) # specify float type
    # reshape it
    acc = np.reshape(acc, (-1, 1))
    # remove extra dimension
    acc = np.squeeze(acc)

    return acc, dt
```

[7]:
```
acc1, dt = load_gm(path1)
acc2, _ = load_gm(path2) # same dt
print(acc1)
```

```
dt = 0.005 sec
dt = 0.005 sec
[6.300641e-04 6.304284e-04 6.307700e-04 … 5.631612e-05 5.655605e-05
 5.678991e-05]
```

3 Simple plotting

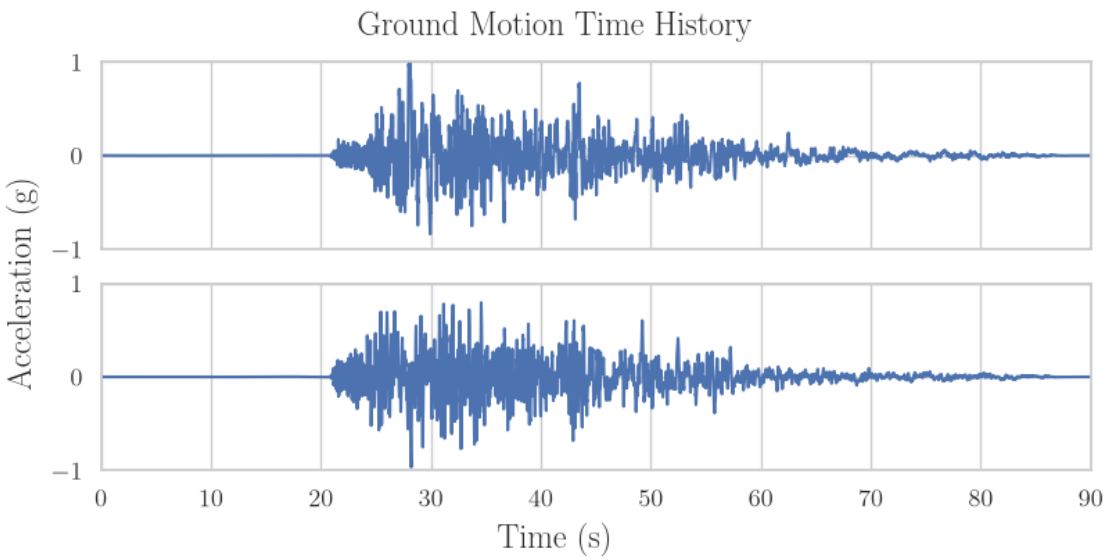
[8]:
```
def increment_all(values):
    for i in range(len(values)):
        values[i] += 1
values = [1,2,3]
increment_all(values)
print(values)
```

```
[2, 3, 4]
```

[9]:
```
plt.close("timehistory")
fig, ax = plt.subplots(2,1, num="timehistory", figsize=(8,4),␣
 ↪layout="constrained", sharex=True, sharey=True)

# setup time axes
t = np.arange(0, len(acc1)*dt, dt) # start, stop, step

ax[0].plot(t, acc1)
ax[1].plot(t, acc2)

ax[0].set_xlim(0, len(acc1)*dt)
ax[0].set_ylim(-1, 1)

fig.supxlabel("Time (s)")
```

```
fig.supylabel("Acceleration (g)")
fig.suptitle("Ground Motion Time History")
```

[9]:
```
Text(0.5, 0.98, 'Ground Motion Time History')
```



[10]:
```
# Integrate twice to get displacements
from scipy import integrate

def integrate_acc(acc:"N x 1 numpy", dt=0.005):
    vel = integrate.cumulative_trapezoid(acc, dx=dt, initial=0)
    disp = integrate.cumulative_trapezoid(vel, dx=dt, initial=0)
    return disp


def rotate(data:"2 x N", theta:"deg"):
    # Rotates data counter clockwise by theta
    t = np.radians(theta)
    R = np.array([[np.cos(t), -np.sin(t)],
                  [np.sin(t),  np.cos(t)]])
    # Multiply matrices
    rotated = R @ data # result is 2 x N
    return rotated
```

[11]:
```
# Test the rotation method
e1 = np.array([[1, 0]]).T
print(e1.shape)
print(rotate(e1, 5))
```

```
(2, 1)
```

```
[[0.9961947 ]
 [0.08715574]]
```

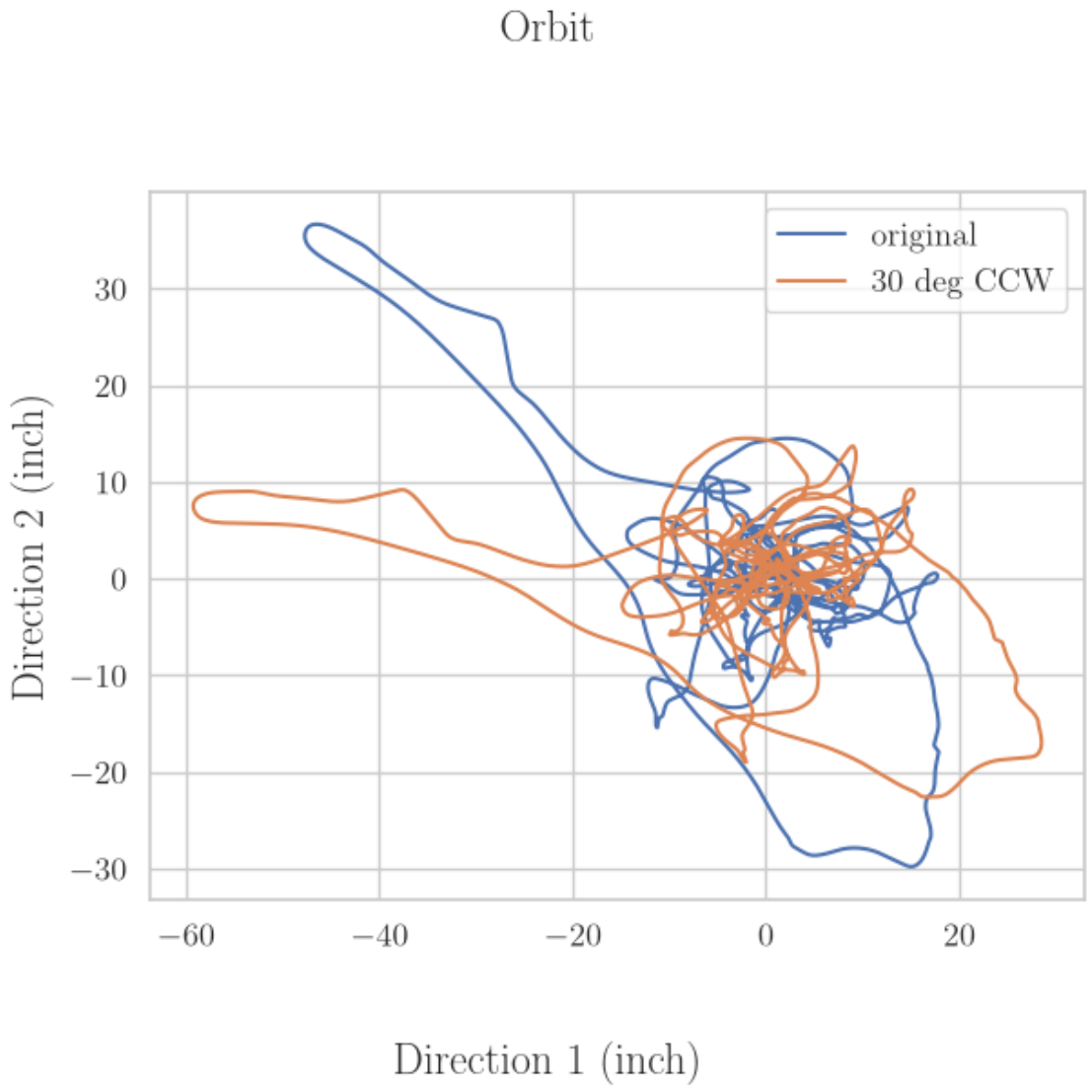
[12]:
```
# Integrate accelerations and rotate
disp1 = integrate_acc(acc1 * 386.1) # convert to in/s^2
disp2 = integrate_acc(acc2 * 386.1)
```

[13]:
```
combined = np.vstack( (disp1, disp2))
print(combined.shape) # 2 x N
rotated = rotate(combined, theta = 30)
print(rotated)
```

```
(2, 18000)
[[ 0.00000000e+00  2.39263025e-06  9.57069736e-06 …  5.07140078e-03
   5.07371641e-03  5.07622269e-03]
 [ 0.00000000e+00  1.93929477e-06  7.76028047e-06 … -1.20808221e-03
  -1.20946379e-03 -1.21008381e-03]]
```

[14]:
```
plt.close("orbit")
fig, ax = plt.subplots(1,1, num="orbit", figsize=(6,6), layout="constrained")

ax.plot(disp1, disp2, label="original")
ax.plot(rotated[0], rotated[1], label="30 deg CCW")

ax.set_aspect('equal', 'box') # Set equal aspect ratio, box shape

ax.legend()

fig.supxlabel("Direction 1 (inch)")
fig.supylabel("Direction 2 (inch)")
fig.suptitle("Orbit")
```

[14]:
```
Text(0.5, 0.98, 'Orbit')
```



[15]:
```
# Save the rotated to a txt file
np.savetxt("data/rotated_displacements.txt", rotated.T, fmt="%.8e", delimiter =␣
↪' ')
```

4 Newton Rhapson exercise

[16]:
```
# Define f(x) and the derivative using methods
def f(x):
    return np.sqrt(x) - 5

def dfdx(x):
    return 1/(2*np.sqrt(x))
```

[17]:
```
# Define using lambda functions
# Define function to solve the root for
f = lambda x: np.sqrt(x) - 5

# Define derivative
dfdx = lambda x: 1/(2*np.sqrt(x))

# Define initial guess
x0 = 1
```

[18]:
```
# Implement newton's method recursively

# Define tolerances with default parameters, add some parameter descriptions
def newton(x0:"initial guess", f, dfdx, max_iter = 10, tol = 1e-5) -> "root␣
 ↪xstar such that f(xstar = 0)":
    # calculate error
    error = f(x0)
    print(f"Iterations left: {max_iter}. Error: {error}")

    # base case, stop the loop
    if abs(error) <= tol or max_iter == 0:
        print(f"Root found at x = {x0}")
        return x0

    # Else, we iterate:

    # calculate next step
    x1 = x0 - f(x0)/dfdx(x0)
    # Call recursively
    return newton(x1, f, dfdx, max_iter - 1, tol=tol)


xstar = newton(x0, f, dfdx)
```

```
Iterations left: 10. Error: -4.0
Iterations left: 9. Error: -2.0
Iterations left: 8. Error: -0.41742430504416017
Iterations left: 7. Error: -0.017454771950544234
Iterations left: 6. Error: -3.0466999208833556e-05
Iterations left: 5. Error: -9.282352664286009e-11
Root found at x = 24.999999999071765
```

[19]:
```
# Using a for loop instead
def newton_forloop(x_initial, f, dfdx, max_iter = 10, tol = 1e-5):
    # initialize
    x0 = x_initial
    for i in range(max_iter):
```

```
        x1 = x0 - f(x0)/dfdx(x0)
        # calculate error
        error = f(x1)
        print(f"Iteration {i}. Error: {error}")

        if abs(error) <= tol:
            # close enough, stop
            print(f"Root found at x = {x1}")
            return x1
        else:
            # update variables and do another loop
            x0 = x1

xstar = newton_forloop(x0, f, dfdx)
```

```
Iteration 0. Error: -2.0
Iteration 1. Error: -0.41742430504416017
Iteration 2. Error: -0.017454771950544234
Iteration 3. Error: -3.0466999208833556e-05
Iteration 4. Error: -9.282352664286009e-11
Root found at x = 24.999999999071765
```

[20]:
```
# Try a harder function
f = lambda x: np.sin(x) + x**2 - np.exp(x)
dfdx = lambda x: np.cos(x) + 2*x - np.exp(x)

xstar = newton(x0, f, dfdx)
```

```
Iterations left: 10. Error: -0.8768108436511486
Iterations left: 9. Error: 16.104195068931052
Iterations left: 8. Error: 3.184305141487436
Iterations left: 7. Error: 0.6439745223762494
Iterations left: 6. Error: 0.07124956256001402
Iterations left: 5. Error: 0.0013753951012600574
Iterations left: 4. Error: 5.508468177151116e-07
Root found at x = -1.1067431155112037
```